

Microservices, Gateway, Resilience & Security — Answers

Account & Loan Microservices

AccountController.java

```
@RestController
public class AccountController {

    @GetMapping("/accounts/{number}")
    public Map<String, Object> getAccount(@PathVariable String number) {
        Map<String, Object> account = new HashMap<>();
        account.put("number", number);
        account.put("type", "savings");
        account.put("balance", 234343);
        return account;
    }
}
```

LoanController.java

```
@RestController
public class LoanController {

    @GetMapping("/loans/{number}")
    public Map<String, Object> getLoan(@PathVariable String number) {
        Map<String, Object> loan = new HashMap<>();
        loan.put("number", number);
        loan.put("type", "car");
        loan.put("loan", 400000);
        loan.put("emi", 3258);
        loan.put("tenure", 18);
        return loan;
    }
}
```

application.properties (loan-service, port conflict fix)

```
server.port=8081
spring.application.name=loan-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Eureka registration recap

account-service and loan-service: add Eureka Discovery Client dependency, put `@EnableDiscoveryClient` on the main class, set `spring.application.name`, then start eureka-discovery-server first, then the services — they show up automatically at `http://localhost:8761`.

Edge Services & API Gateway (Routing, Load Balancing, Resilience)

1. Routing & Filtering

```
spring.cloud.gateway.routes[0].id=example_route
spring.cloud.gateway.routes[0].uri=http://example.org
spring.cloud.gateway.routes[0].predicates[0]=Path=/example/**

@Component
public class LoggingFilter implements GlobalFilter {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        System.out.println("Request: " + exchange.getRequest().getURI());
    }
}
```

```

        return chain.filter(exchange);
    }
}

```

2. Load Balancing

```

spring.cloud.gateway.routes[0].id=load_balanced_route
spring.cloud.gateway.routes[0].uri=lb://example-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/loadbalanced/**

```

lb:// tells the gateway to resolve example-service via the LoadBalancerClient instead of hitting a fixed host — Eureka gives it the list of live instances to round-robin/random across.

3. Resilience (Circuit Breaker)

```

resilience4j.circuitbreaker.instances.exampleCircuitBreaker.slidingWindowSize=10
resilience4j.circuitbreaker.instances.exampleCircuitBreaker.failureRateThreshold=50

```

Once failureRateThreshold (50%) is crossed within the last 10 calls (slidingWindowSize), the breaker trips to OPEN and short-circuits further calls instead of letting them hang/timeout.

Spring Boot 3.0 Microservices Exercises

4. User & Order Management System

```

// User Service
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private String email;
}

@RestController
@RequestMapping("/users")
public class UserController {
    @Autowired private UserRepository userRepository;

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userRepository.findById(id).orElseThrow();
    }
}

// Order Service - calls User Service via OpenFeign
@FeignClient(name = "user-service")
public interface UserClient {
    @GetMapping("/users/{id}")
    User getUser(@PathVariable("id") Long id);
}

@Entity
public class Order {
    @Id @GeneratedValue
    private Long id;
    private Long userId;
    private String product;
    private Double amount;
}

@RestController
@RequestMapping("/orders")
public class OrderController {

```

```

    @Autowired private OrderRepository orderRepository;
    @Autowired private UserClient userClient;

    @PostMapping
    public Order placeOrder(@RequestBody Order order) {
        User user = userClient.getUser(order.getUserId()); // validates user exists
        return orderRepository.save(order);
    }
}

```

application.properties for both: spring.datasource.url pointing to MySQL/Postgres, spring.jpa.hibernate.ddl-auto=update. Order Service additionally needs @EnableFeignClients on its main class.

5. Inventory Management with Service Discovery

```

// Config Server main class
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { }

// application.properties (config server)
server.port=8888
spring.cloud.config.server.git.uri=file:///D:/microservices/config-repo

// Product Service / Inventory Service - bootstrap.properties
spring.application.name=product-service
spring.cloud.config.uri=http://localhost:8888
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

@RestController
@RequestMapping("/inventory")
public class InventoryController {
    @Autowired private InventoryRepository inventoryRepository;

    @GetMapping("/{productId}")
    public Inventory getStock(@PathVariable Long productId) {
        return inventoryRepository.findByProductId(productId);
    }
}

```

API Gateway with Rate Limiting, Caching, Path Rewriting

```

spring.cloud.gateway.routes[0].id=customer-service
spring.cloud.gateway.routes[0].uri=lb://customer-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/api/customers/**
spring.cloud.gateway.routes[0].filters[0]=RewritePath=/api/customers/(?
<segment>.*), /${segment}
spring.cloud.gateway.routes[0].filters[1]=RequestRateLimiter.args.redis-rate-
limiter.replenishRate=5
spring.cloud.gateway.routes[0].filters[1]=RequestRateLimiter.args.redis-rate-
limiter.burstCapacity=10

spring.cloud.gateway.routes[1].id=billing-service
spring.cloud.gateway.routes[1].uri=lb://billing-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/api/billing/**

@Bean
public KeyResolver userKeyResolver() { return
    exchange -> Mono.just(
        exchange.getRequest().getRemoteAddress().getAddress().getHostAddress());
}

```

Rate limiting needs spring-boot-starter-data-redis-reactive on the classpath since RequestRateLimiter is backed by Redis. Caching can be added via spring-cloud-starter-gateway's built-in LocalResponseCache filter (cache-request-body / response-cache) on GET routes.

6. Resilient Microservices with Circuit Breaker

```
@Service
public class PaymentService {

    private static final Logger LOGGER =
LoggerFactory.getLogger(PaymentService.class);

    @Autowired
    private RestTemplate restTemplate;

    @CircuitBreaker(name = "paymentApi", fallbackMethod = "paymentFallback")
    public String callThirdPartyApi() {
        return restTemplate.getForObject("https://slow-third-party.example.com/pay",
String.class);
    }

    public String paymentFallback(Throwable t) {
        LOGGER.warn("Fallback triggered for payment API: {}", t.getMessage());
        return "Payment service temporarily unavailable, please retry shortly";
    }
}

resilience4j.circuitbreaker.instances.paymentApi.slidingWindowSize=10
resilience4j.circuitbreaker.instances.paymentApi.failureRateThreshold=50
resilience4j.circuitbreaker.instances.paymentApi.waitDurationInOpenState=10s
resilience4j.circuitbreaker.instances.paymentApi.registerHealthIndicator=true
```

Centralized Authentication & SSO

7. OAuth2/OIDC Login

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-client:
            client-id: YOUR_CLIENT_ID
            client-secret: YOUR_CLIENT_SECRET
            scope: openid, profile, email
            authorization-grant-type: authorization_code

@RestController
public class UserController {
    @GetMapping("/user")
    public Principal user(Principal principal) {
        return principal;
    }
}
```

8. Resource Server (validates JWT from an Authorization Server)

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://issuer.example.com

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
}
```

```
    return http.build();  
}
```

9. Custom JWT Filter

```
public class JwtTokenFilter extends OncePerRequestFilter {  
  
    @Autowired  
    private JwtTokenProvider jwtTokenProvider;  
  
    @Override  
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse  
response,  
        FilterChain filterChain) throws ServletException, IOException {  
  
        String token = resolveToken(request);  
        if (token != null && jwtTokenProvider.validateToken(token)) {  
            Authentication auth = jwtTokenProvider.getAuthentication(token);  
            SecurityContextHolder.getContext().setAuthentication(auth);  
        }  
        filterChain.doFilter(request, response);  
    }  
  
    private String resolveToken(HttpServletRequest request) {  
        String bearerToken = request.getHeader("Authorization");  
        if (bearerToken != null && bearerToken.startsWith("Bearer ")) {  
            return bearerToken.substring(7);  
        }  
        return null;  
    }  
}
```

Register it with `.addFilterBefore(jwtTokenFilter, UsernamePasswordAuthenticationFilter.class)` in the `SecurityFilterChain` so every request is checked for a valid token before Spring's default auth filter runs.